



空间数据库原理与应用

第九讲：空间数据库查询（一）

任课教师：李晓明

建筑与城市规划学院 城市空间信息工程系

其中部分图片来自互联网和同行专家

目录

CONTENTS

01

空间查询回顾

02

空间查询语言

03

PostGIS的空间查询

目录

CONTENTS

1

空间查询回顾

1.1 空间查询定义

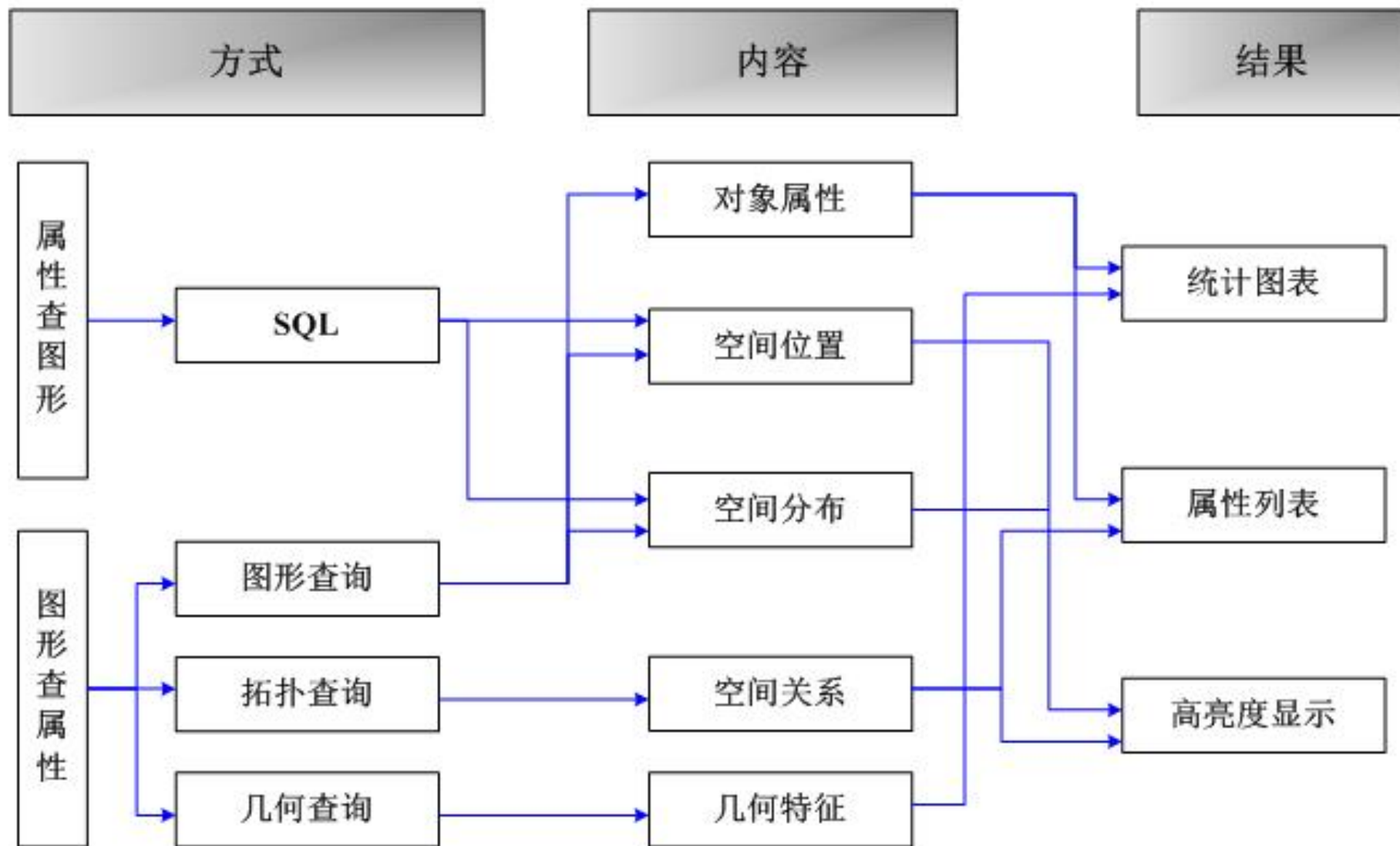
1.2 空间查询方式

1.1 空间查询定义

- **定义：从空间数据库中查找出满足某一条件的空间目标的过程。**
- 在GIS中，空间查询无处不在，是其它操作（增加、删除、更新等）的基础
- 空间查询性能是GIS应用系统成功的关键所在



1.2 空间查询方式



空间数据查询的方式、内容与结果

目录

CONTENTS

2

空间查询语言

2.1 标准的SQL查询语言

2.2 扩展的SQL查询语言

2.1 标准的SQL查询语言

SQL是关系数据库管理系统（RDBMS）的标准查询语言，是一种声明性语言，即用户只需描述所要的结果即可，而不必描述获得结果的过程。

SQL语句由命令、从句、运算符、合计函数等构成。SQL的功能包括：查询、操作、定义、控制，是一个综合的、通用的、功能极强的关系数据库语言。

SELECT语句一般格式：

SELECT[ALL|DISTINCT]<目标类表达式>[别名]...

FROM<表名或视图名>[别名][,<表名或视图名>[别名]].....

[**WHERE**<条件表达式>]

[**GROUP BY** <列名1>[**HAVING**<条件表达式>]]

[**ORDER BY** <别名2>[ASC|DESC]];

2.1 标准的SQL查询语言

- **SQL语言的不足**

- 1) 数据类型太简单，只有整型、日期型、字符串型等，不能支持点线面等复杂数据类型（没有相应的操作原语）。
- 2) 所有数据类型都是固定的，无法增加新的数据类型。

- **解决方法：**

- 1) 利用BLOB类型字段存储空间信息，依赖于宿主语言的应用程序代码。
- 2) 混合系统，空间数据以文件形式保存在数据库外部（比如，MapInfo）。无法利用数据库的服务，比如查询语言、并发控制、索引支持等。
- 3) 利用OR-DBMS自定义ADT。比如，自定义polygon数据类型，以及相应的函数方法，比如adjacent。

■ 2.2 扩展的SQL查询语言

扩展的SQL查询语言主要类型

- 1、查询谓词的扩展
- 2、面向对象的扩展
- 3、模糊扩展

2.2 扩展的SQL查询语言

扩展的SQL查询语言主要类型

1、查询谓词的扩展

- 增加空间数据类型 （点、线、面）
- 增加空间操作算子 （长度、面积）
- 增加查询条件 （临近、叠加、经过）

2.2 扩展的SQL查询语言

扩展的SQL查询语言主要类型

1、查询谓词的扩展

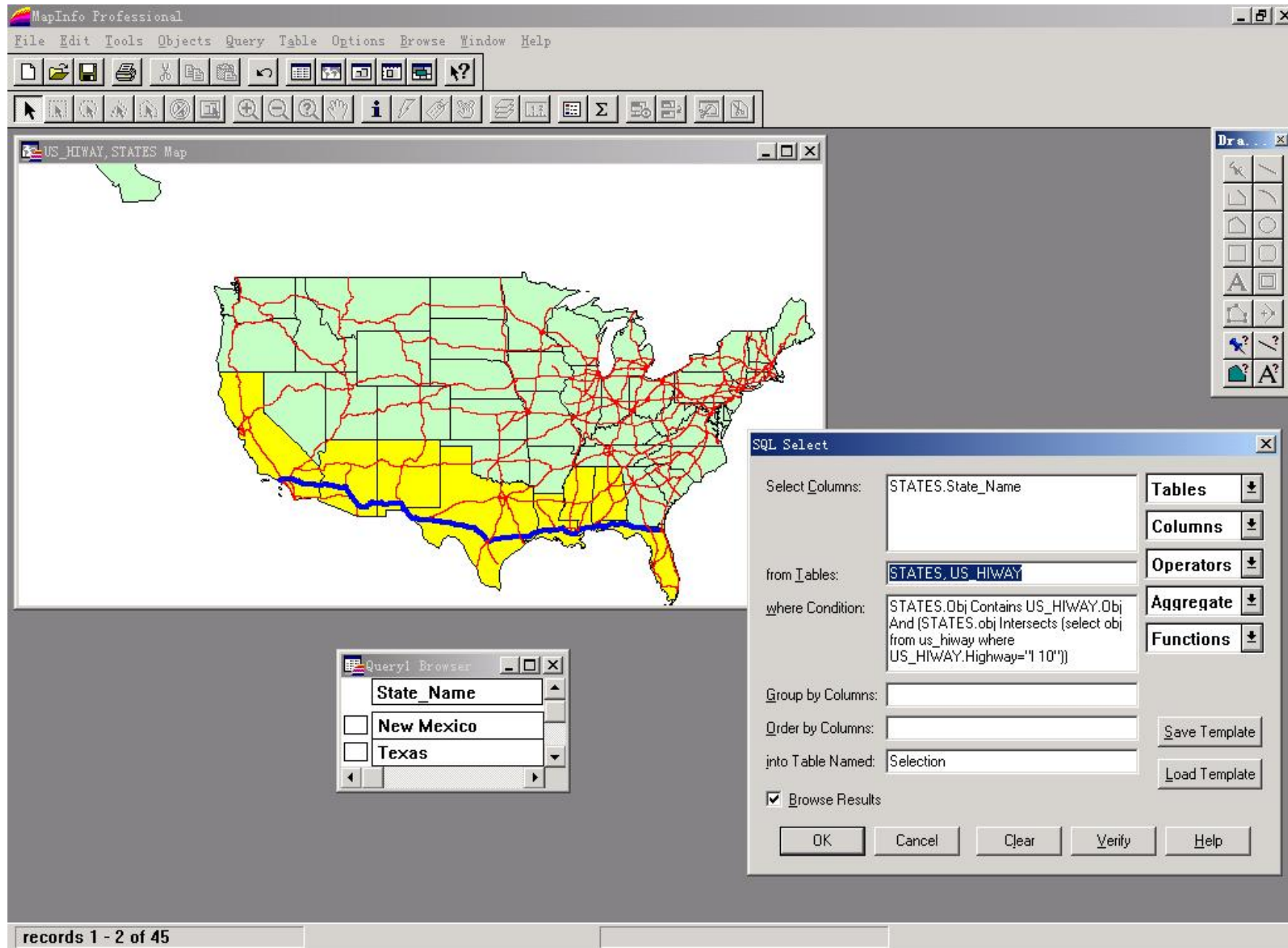
Mapinfo在SELECT语句中增加了**地理函数**和**地理运算符**.

例：美国 “I 10”号高速公路经过哪几个洲？

- 先美国高速公路中找出 “I10”号高速公路；
- 再找 “I 10”号高速公路经过哪几个洲。

地 理 运 算 符

WHERE States.obj **CONTAINS** Us_Hiway.obj AND
(States.obj **INTERSECTS** (SELECT obj FROM Us_Hiway
WHERE us_Hiway.highway= “I 10”))



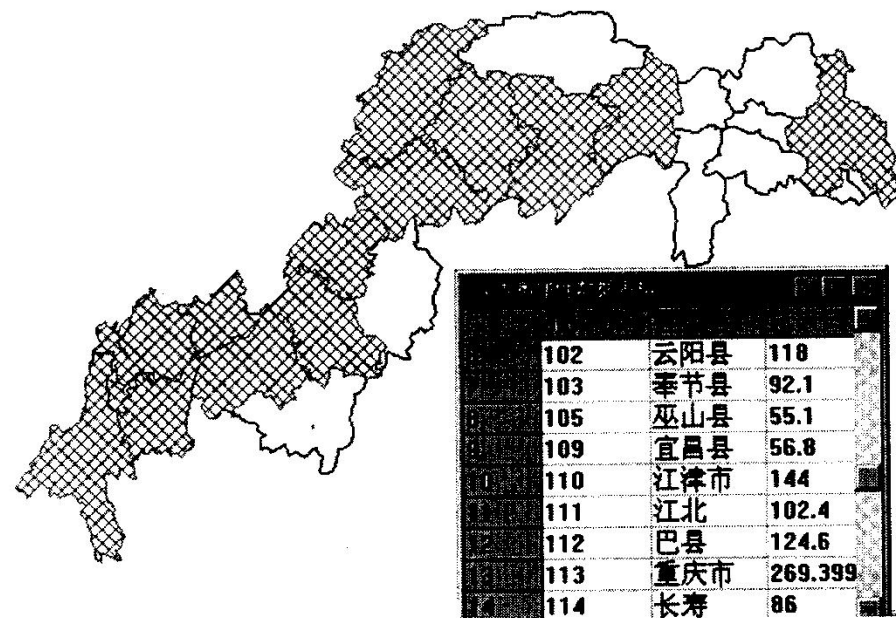
2.2 扩展的SQL查询语言

扩展的SQL查询语言主要类型

1、查询谓词的扩展

例如查询三峡地区长江流域人口大于50万的县或市，扩展的SQL空间查询语句为：

```
SELECT *
FROM 县或市
WHERE 县或市.人口>50万
AND CROSS (河流.名称="长江" )
```



2.2 扩展的SQL查询语言

扩展的SQL查询语言主要类型

1、查询谓词的扩展

这些SQL扩充和应用有关,跟GIS软件支持程度关系较大。

例:

(1) 选择河南省所有城市和人口

```
SELECT 城市名, 人口 FROM 城市  
WHERE CENTER (城市地图) INSIDE 河南;
```

(2) 选择流经河南省的所有河流的名称和河南境内长度

```
SELECT 河流名, LENGTH (INTERSECTS (ROUTE (河流流域图), 河南));  
FROM 河流 WHERE ROUTE (河流流域图) INTERSECTS 河南;
```

2.2 扩展的SQL查询语言

扩展的SQL查询语言主要类型

2、面向对象扩展

**采用面向对象的方法来设计SQL语言（OOSQL）。
优点如下：**

- (1) 良好的查询机制，易于实现持久性。**
- (2) 对象简化了查询，解决了某些常见的 SQL难题。**

2.2 扩展的SQL查询语言

扩展的SQL查询语言主要类型

2、面向对象扩展

- OGIS协会（Open GIS）是由一些主要软件供应商组成的联盟，负责制定与GIS互操作相关的行标准。OGIS的空间数据模型可以嵌入到各种编程语言中，例如C、Java、SQL等等，提出了一套规范，把二维地理空间ADT（abstract data type, 抽象数据类型）整合到SQL之中，并且包括了指定拓扑的操作和空间分析操作。
- 面向对象数据库系统对扩展DBMS的功能来支持空间（复杂）对象，扩展了关系数据库的面向对象特性，最终产生对象关系数据库管理系统（OR-DBMS）的通用框架。

2.2 扩展的SQL查询语言

扩展的SQL查询语言主要类型

3、模糊扩展

对查询谓词的空间扩展只能处理精确数据，无法表达自然语言中的模糊概念。
因此，在具体应用中对SQL进行模糊扩展。

例如：查询面积较小，噪声小，住宅低价偏低的宗地号。

```
SELECT 宗地号 FROM table_price
```

```
WHERE (面积='较小' AND 噪声='小' AND 住宅低价='偏低' ) ;
```

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

- **支持四个子空间几何类型：**
 - Point; Curve; Surface; GeometryCollection
- **操作分类：**
 - 1) 用于所有几何类型的基本操作。

例如，SpatialReference返回几何体采用的坐标系统
 - 2) 用于空间对象之间拓扑关系的操作测试。

例如，overlap返回两个对象是否相交。
 - 3) 用与空间分析的一般操作。

例如，distance返回两个空间对象之间的最短距离。

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

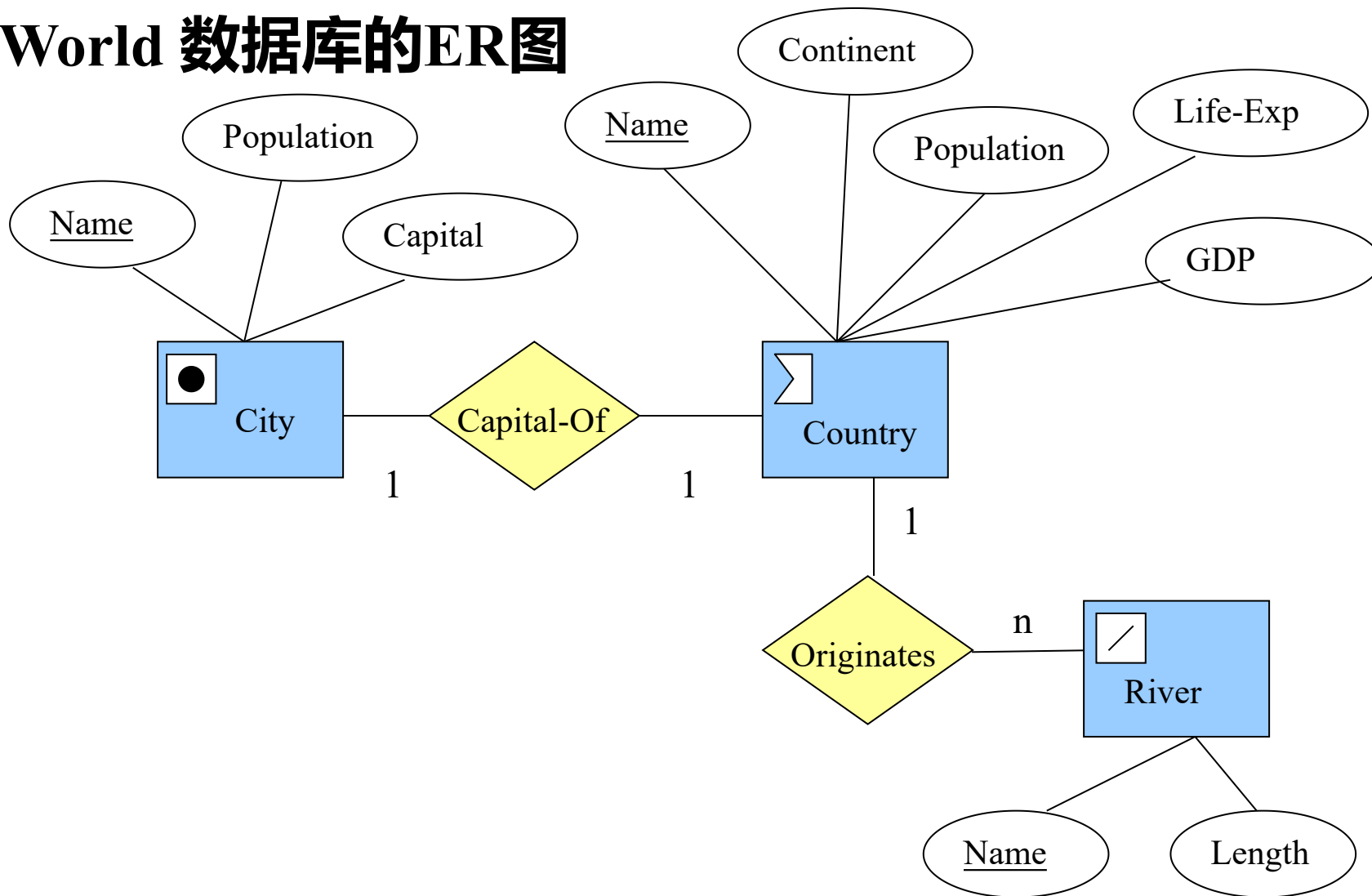
• 基本操作

基本函数	SpatialReference	几何体的坐标系
	Envelope	几何体最小外接矩形
	Export	其他形式的几何体
	IsEmpty	判断几何体是否为空
	IsSimple	判断几何体是否自交
	Boundary	几何体的边界
	Equal	判断两个几何体是否相等
	DisJoint	判断几何体是否相接
	Intersect	判断几何体是否相交
	Touch	判断几何体是否相邻
	Cross	判断线与面是否相交
	Within	判断是否被包含
	Contains	判断是否包含
	Overlap	判断是否有重叠
	Distance	几何体之间最短距离
	Buffer	几何体缓冲区
	ConvexHull	几何体最小凸包
	Intersection	两个几何体的交集
	Union	两个几何体的并集
空间分析	Difference	几何体的差集
	SymmDiff	两个几何体的不相交部分

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

实例：World 数据库的ER图



2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

- World数据库的模式:

Country (*Name*: varchar(35),
Cont: varchar(35),
Pop: Integer,
GDP: Numeric,
Life-Exp: Numeric,
Shape: Polygonid)

City (*Name*: varchar(35),
Country: varchar(35),
Pop: Integer,
Capital: char(1) ,
Shape: Pointid)

River (*Name*: varchar(35),
Origin-Country: varchar(35),
Length: Numeric,
Shape: Lineid)

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

Country表

Name	Cont	Pop (m)	GDP (b)	Life-Exp	Shape
Canada	NAM	30.1	658.0	77.08	Polygonid-1
Mexico	NAM	107.5	694.3	69.36	Polygonid-2
Brazil	SAM	183.3	1004.0	65.60	Polygonid-3
Cuba	NAM	11.7	16.9	75.95	Polygonid-4
USA	NAM	270.0	8003.0	75.75	Polygonid-5
Argentina	SAM	36.3	348.2	70.75	Polygonid-6

City表

Name	Country	Pop (m)	Capital	Shape
Havana	Cuba	2.1	Y	Pointid-1
Washington, D.C	USA	3.2	Y	Pointid-2
Monterrey	Mexico	2.0	N	Pointid-3
Toronto	Canada	3.4	N	Pointid-4
Brasilia	Brazil	1.5	Y	Pointid-5
Ottawa	Canada	0.8	Y	Pointid-6

River表

Name	Origin	Length (km)	Shape
Rio Parana	Brazil	2600	LineStringid-1
St. Lawrence	USA	1200	LineStringid-2
Rio Grande	USA	3000	LineStringid-3
Mississippi	USA	6000	LineStringid-4

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询1：列出Country表中所有与美国相邻的国家

```
SELECT      C1.Name AS “Neighbors of USA ”  
FROM        Country C1, Country C2  
WHERE       Touch (C1.Shape, C2.Shape) = 1  
            AND   C2.Name=“USA”
```

谓词Touch 检测两个几何对象是否彼此相邻而又不交叠。OGIS中定义的8个拓扑谓词之一。

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询2：找出河流流经的国家

```
SELECT      R.Name C.Name  
FROM        River R, Country C  
WHERE       Cross (R.Shape, C.Shape)
```

拓扑谓词Cross常用于线对象与多边形对象的穿过判断。

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询3：对于River表中列出的所有河流，找出距离河流最近的城市。

```
SELECT      C1.Name R1.Name
FROM        City C1, River R1
WHERE       Distance (C1.Shape, R1.Shape) <
           ALL (
             SELECT Distance (C2.Shape, R1.Shape)
             FROM City C2
             WHERE  C1.Name<>C2.Name )
```

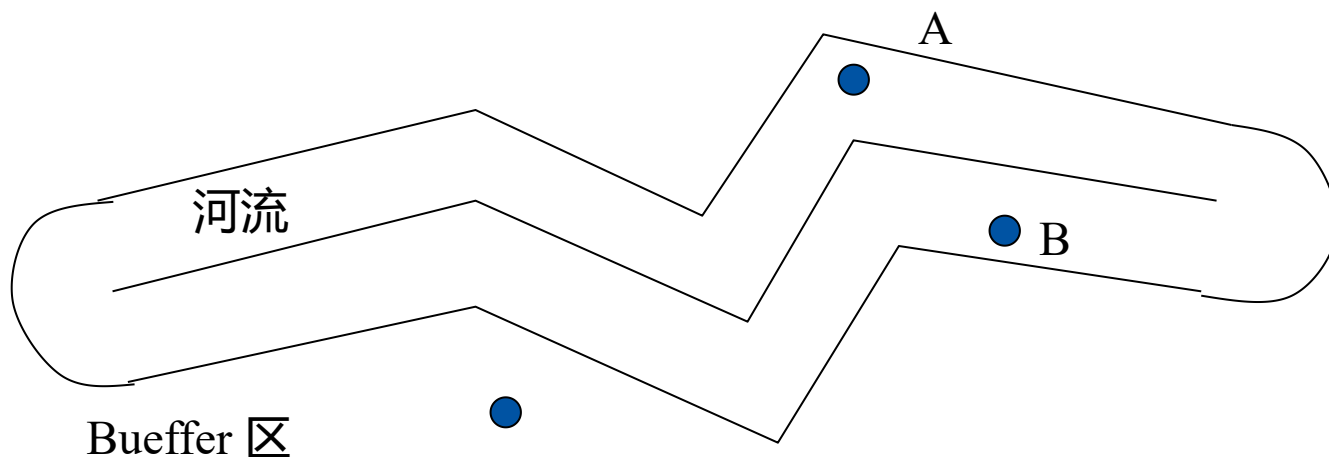
Distance 一个 二元运算，返回实数值。

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询4：圣劳伦斯河能为方圆300公里以内的城市供水，列出能从该河获得供水的城市。

```
SELECT    Ci.Name
FROM      City Ci, River R
WHERE     Overlap(Ci.Shape, Buffer(R.Shape, 300))=1 AND
          R.Name = "St.Lawrence"
```



2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询5：列出Country表中每个国家的名字、人口和国土面积

```
SELECT      C.Name, C.Pop, Area(C.Shape) AS “Area”  
FROM        Country C
```

面积函数 Area, 只适用于Polygon 和 MultiPolygon

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询6： 求出河流在流经的各国境内的长度

```
SELECT      R.Name, C.Name, Length (Intersection (R.Shape, C.Shape))  
            AS "Length"  
FROM        River R, Country C  
WHERE       Cross (R.Shape, C.Shape ) =1
```

Intersection () 函数计算两个几何对象的交部分。
Intersects 是一个拓扑谓词。

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询7：列出每个国家的GDP以及其首都到赤道的距离。

```
SELECT      Co.GDP, Distance ( Point( 0, Ci.Shape.x), Ci.Shape)
            AS “Distance”
FROM        Country Co, City Ci
WHERE       Co.Name=Ci.Country AND
            Ci.Captial = “Y”
```

联机分析处理 OLAP, 数据挖掘

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询8：按其邻国数目的多少列出所有国家

```
SELECT      Co.Name, Count (Co1.Name)
FROM        Country Co, Country Co1
WHERE       Touch ( Co.Shape, Co1.Shape )
            GROUP BY Co.Name
            GROUP BY Count(Co1.Name)
```

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

查询9：哪一个国家的邻国最多

```
CREATE VIEW Neighbor AS
SELECT Co.Name, Count (Co1.Name) AS num-neighbors
FROM Country Co, Country Co1
WHERE Touch ( Co.Shape, Co1.Shape )
GROUP BY Co.Name

SELECT Co.Name, num_neighbors
FROM Neighbor
WHERE num_neighbor = (SELECT Max (num_neighbors)
FROM Neighbor )
```

2.2 扩展的SQL查询语言

OGIS的SQL空间扩展

- **OGIS标准的局限性：**
 - 1) 支持空间对象模型，对场模型支持不好。
 - 2) 对象模型查询操作存在局限性。（GROUP BY , HAVING存在问题）
 - 3) 不支持基于方位的谓词操作（比如，东南西北，上下左右前后等）
 - 4) 不支持基于形状、基于可见性等的操作。

2.2 扩展的SQL查询语言

对象关系SQL

- ❑ OGIS定义了数据类型及其相关运算，对空间应用这些是必不可少的，但数据类型和运算之间存在限制。
- ❑ 对于对象关系数据库或面向对象数据库数据类型不存在限制，这些数据库支持用户自定义数据类型。这种特性显然是一个优点，尤其是处理GIS这类非传统数据库应用时更加有用。
- ❑ 数据库应用开发人员在构建符合语法和语义的数据类型方面的负担则更重了。商业数据库厂商推出了特殊应用的“软件包”，给数据库用户提供无缝接口。例如，Oracle就有专为GIS领域开发的软件包—Spatial Data Cartridges。
- ❑ 针对OR-DBMS提出的SQL标准SQL3/SQL99可以使用户在关系数据库的框架内定义自己的数据类型。

2.2 扩展的SQL查询语言

对象关系SQL

对象关系模式

- Oracle 9i 是Oracle 公司开发的对象-关系数据库管理系统（OR-DBMS）；
- OR-DBMS实现了SQL3标准的一部分，ADT在这种系统中称作“对象类型”（Object Type）

2.2 扩展的SQL查询语言

对象关系SQL

SQL3概览

- SQL3/SQL99主要在以下两方面进行扩展：
- 1) ADT：可以使用CREATE TYPE语句来定义ADT。ADT由一组属性和访问这些属性值的成员函数组成，成员函数可以隐含地修改数据类型中的属性值，也就能够改变数据库的状态。例如用ADT定义列属性的类型：

```
CREATE TYPE Point (  
  x NUMBER,  
  y NUMBER  
  FUNCTION Distance (: u Point, : v Point)  
  RETURNS NUMBER)
```

- u和v前的冒号表示这两个变量是局部变量。

2.2 扩展的SQL查询语言

对象关系SQL

SQL3概览

- 2) 行类型：行类型是用于定义关系的类型，它指定了关系的模式。
例如：

```
CREATE ROW TYPE Point (  
  x NUMBER,  
  y NUMBER) ;
```

可以创建一个表作为这个行类型的实例：

```
CREATE TABLE PointTable of TYPE Point;
```

注意：用ADT作为列类型能很自然地与OR-DBMS保持一致

2.2 扩展的SQL查询语言

对象关系SQL

- 建立点对象

```
CREATE TYPE Point AS OBJECT (  
    x NUMBER,  
    y NUMBER,  
    MEMBER FUNCTION Distance ( P2 IN Point ) RETURN NUMBER  
    PRAGMA RESTRICT_REFERENCES ( Distance, WNDS));  
.....
```

- City的对象-关系模式可定义为

```
CREATE TABLE City (  
    Name varchar(30)  
    Country varchar(30)  
    Pop Number  
    Shape Point );
```

2.2 扩展的SQL查询语言

对象关系SQL

Oracle 中构造基本空间对象 LineString

- 首先创建一个中间简单类型 LineType

```
CREATE TYPE LineType AS VARRAY (500) OF Point
```

- 创建线对象LineString (ADT)

```
CREATE TYPE LineString AS OBJECT (  
    Num_of_Points INT  
    Geometry LineType  
    MEMBER FUNCTION Length (SELF IN) RETURN NUMBER,  
    PRAGMA RESTRICT_REFERENCES(LENGTH, WNDS));
```

2.2 扩展的SQL查询语言

对象关系SQL

创建河流的例子

```
CREATE TYPE River (  
    Name Varchar (30)  
    Origin Varchar(30)  
    Length number,  
    Shape LineString );
```

```
INSERT INTO RIVER (“MISSISSIPPI”, “USA”, 6000, LineString(3,  
LineType(Point(1,1), Point(1,2), Point(2,3))))
```

2.2 扩展的SQL查询语言

对象关系SQL

Oracle 中构造基本空间对象Polygon

```
CREATE TYPE PolyType AS VARRAY (1000) OF Point
```

```
CREATE TYPE Polygon AS OBJECT (  
    Num_of_Points INT,  
    Geometry PolyType,  
    MEMBER FUNCTION Area(SELF IN) RETURN NUMBER,  
    PRAGMA RESTRICT_REFERENCES(Area, WNDS));
```


2.2 扩展的SQL查询语言

对象关系SQL

利用Polygon 建立国家表格

```
CREATE TABLE Country (
```

```
    Name    Varchar (30)
```

```
    Cont    Varchar (30)
```

```
    Pop     int,
```

```
    GDP     number,
```

```
    Life-Exp number,
```

```
    Shape   Polygon );
```

```
INSERT INTO Country ( "Mexico", "NAM", 107.5, 694.3, 69.36, Polygon(23,  
Polytype(Point(1,1) ,.....,Point(1,1)))
```

2.2 扩展的SQL查询语言

对象关系SQL

查询示例

找出City表列出的所有城市两两之间的距离

```
SELECT  C1.Name, C1.Distance(C2.Shape) AS "Distance"  
FROM    City C1, City C2  
WHERE   C1.Name <> C2.Name
```

列出与美国接壤的国家的名字、人口和国土面积

```
SELECT  C2.Name, C2.Pop, C2.Area() AS "Area"  
FROM    Country C1, Country C2  
WHERE   C1.Name = "USA" AND  
        C1.Touch(C2.Shape) = 1
```

ADT函数

拓扑谓词

目录

CONTENTS

3

PostGIS的空间查询

3.1 PostGIS空间函数

3.2 PostGIS空间查询语言

3.1 PostGIS空间函数

1. OGC标准函数

管理函数：

添加几何字段 `AddGeometryColumn(, , , , )`

删除几何字段 `DropGeometryColumn(, , )`

检查数据库几何字段并在geometry_columns中归档 `Probe_Geometry_Columns()`

给几何对象设置空间参考（在通过一个范围做空间查询时常用） `ST_SetSRID(geometry, integer)`

3.1 PostGIS空间函数

1. OGC标准函数

几何对象关系函数：

获取两个几何对象间的距离 ST_Distance(geometry, geometry)

如果两个几何对象间距离在给定值范围内，则返回TRUE ST_DWithin(geometry, geometry, float)

判断两个几何对象是否相等

(比如LINESTRING(0 0, 2 2)和LINESTRING(0 0, 1 1, 2 2)是相同的几何对象) ST_Equals(geometry, geometry)

判断两个几何对象是否分离 ST_Disjoint(geometry, geometry)

判断两个几何对象是否相交 ST_Intersects(geometry, geometry)

判断两个几何对象的边缘是否接触 ST_Touches(geometry, geometry)

判断两个几何对象是否互相穿过 ST_Crosses(geometry, geometry)

判断A是否被B包含 ST_Within(geometry A, geometry B)

判断两个几何对象是否是重叠 ST_Overlaps(geometry, geometry)

判断A是否包含B ST_Contains(geometry A, geometry B)

判断A是否覆盖 B ST_Covers(geometry A, geometry B)

判断A是否被B所覆盖 ST_CoveredBy(geometry A, geometry B)

通过DE-9IM 矩阵判断两个几何对象的关系是否成立 ST_Relate(geometry, geometry, intersectionPatternMatrix)

获得两个几何对象的关系 (DE-9IM矩阵) ST_Relate(geometry, geometry)

3.1 PostGIS空间函数

1. OGC标准函数

几何对象处理函数:

获取几何对象的中心 ST_Centroid(geometry)

面积量测 ST_Area(geometry)

长度量测 ST_Length(geometry)

返回曲面上的一个点 ST_PointOnSurface(geometry)

获取边界 ST_Boundary(geometry)

获取缓冲后的几何对象 ST_Buffer(geometry, double, [integer])

获取多几何对象的外接对象 ST_ConvexHull(geometry)

获取两个几何对象相交的部分 ST_Intersection(geometry, geometry)

将经度小于0的值加360使所有经度值在0-360间 ST_Shift_Longitude(geometry)

获取两个几何对象不相交的部分 (A、B可互换) ST_SymDifference(geometry A, geometry B)

从A去除和B相交的部分后返回 ST_Difference(geometry A, geometry B)

返回两个几何对象的合并结果 ST_Union(geometry, geometry)

返回一系列几何对象的合并结果 ST_Union(geometry set)

用较少的内存和较长的时间完成合并操作, 结果和ST_Union相同 ST_MemUnion(geometry set)

3.1 PostGIS空间函数

1. OGC标准函数

几何对象存取函数:

获取几何对象的WKT描述 ST_AsText(geometry)

获取几何对象的WKB描述 ST_AsBinary(geometry)

获取几何对象的空间参考ID ST_SRID(geometry)

获取几何对象的维数 ST_Dimension(geometry)

获取几何对象的边界范围 ST_Envelope(geometry)

判断几何对象是否为空 ST_IsEmpty(geometry)

判断几何对象是否不包含特殊点（比如自相交） ST_IsSimple(geometry)

判断几何对象是否闭合 ST_IsClosed(geometry)

判断曲线是否闭合并且不包含特殊点 ST_IsRing(geometry)

获取多几何对象中的对象个数 ST_NumGeometries(geometry)

获取多几何对象中第N个对象 ST_GeometryN(geometry,int)

获取几何对象中的点个数 ST_NumPoints(geometry)

获取几何对象的第N个点 ST_PointN(geometry,integer)

3.1 PostGIS空间函数

1. OGC标准函数

几何对象存取函数:

获取多边形的外边缘 ST_ExteriorRing(geometry)

获取多边形内边界个数 ST_NumInteriorRings(geometry)

同上 ST_NumInteriorRing(geometry)

获取多边形的第N个内边界 ST_InteriorRingN(geometry,integer)

获取线的终点 ST_EndPoint(geometry)

获取线的起始点 ST_StartPoint(geometry)

获取几何对象的类型 GeometryType(geometry)

类似上，但是不检查M值，即POINTM对象会被判断为point

ST_GeometryType(geometry)

获取点的X坐标 ST_X(geometry)

获取点的Y坐标 ST_Y(geometry)

获取点的Z坐标 ST_Z(geometry)

获取点的M值 ST_M(geometry)

3.1 PostGIS空间函数

1. OGC标准函数

几何对象构造函数：

参考语义：

Text: WKT

WKB: WKB

Geom: Geometry

M: Multi

Bd: BuildArea

Coll: Collection ST_GeomFromText(text,[])

```
ST_PointFromText(text,[])  
ST_LineFromText(text,[])  
ST_LineStringFromText(text,[])  
ST_PolyFromText(text,[])  
ST_PolygonFromText(text,[])  
ST_MPointFromText(text,[])  
ST_MLineFromText(text,[])  
ST_MPolyFromText(text,[])  
ST_GeomCollFromText(text,[])  
ST_GeomFromWKB(bytea,[])  
ST_GeometryFromWKB(bytea,[])  
ST_PointFromWKB(bytea,[])  
ST_LineFromWKB(bytea,[])  
ST_LineStringFromWKB(bytea,[])  
ST_PolyFromWKB(bytea,[])  
ST_PolygonFromWKB(bytea,[])  
ST_MPointFromWKB(bytea,[])  
ST_MLineFromWKB(bytea,[])  
ST_MPolyFromWKB(bytea,[])  
ST_GeomCollFromWKB(bytea,[])  
ST_BdPolyFromText(text WKT, integer SRID)  
ST_BdMPolyFromText(text WKT, integer SRID)
```

3.1 PostGIS空间函数

2. PostGIS扩展函数

管理函数：

删除一个空间表（包括geometry_columns中的记录）

DropGeometryTable([],)

更新空间表的空间参考 UpdateGeometrySRID([], , ,)

更新空间表的统计信息 update_geometry_stats([],)

参考语义：

Geos：GEOS库

Jts：JTS库

Proj：PROJ4库 postgis_version()

postgis_lib_version()

postgis_lib_build_date()

postgis_script_build_date()

postgis_scripts_installed()

postgis_scripts_released()

postgis_geos_version()

postgis_jts_version()

postgis_proj_version()

postgis_uses_stats()

postgis_full_version()

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何操作符:

A范围=B范围 $A = B$

A范围覆盖B范围或A范围在B范围左侧 $A \&\lt\> B$

A范围在B范围左侧 $A \<\>> B$

A范围覆盖B范围或A范围在B范围下方 $A \&\lt| B$ A范围覆盖

B范围或A范围在B范围上方 $A | \&\> B$

A范围在B范围下方 $A \<\<| B$ A范围在B范围上方 $A | \>\> B$

$A = B$ $A \sim = B$

A范围被B范围包含 $A @ B$

A范围包含B范围 $A \sim B$

A范围覆盖B范围 $A \&\& B$

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何量测函数:

量测面积 ST_Area(geometry)

根据经纬度点计算在地球表面上的距离, 单位米, 地球半径取值6370986米 ST_distance_sphere(point, point)

类似上, 使用指定的地球椭球参数 ST_distance_spheroid(point, point, spheroid)

量测2D对象长度 ST_length2d(geometry)

量测3D对象长度 ST_length3d(geometry)

根据经纬度对象计算在地球表面上的长度 ST_length_spheroid(geometry,spheroid)

ST_length3d_spheroid(geometry,spheroid)

量测两个对象间距离 ST_distance(geometry, geometry)

量测两条线之间的最大距离 ST_max_distance(linestring,linestring)

量测2D对象的周长 ST_perimeter(geometry)

ST_perimeter2d(geometry)

量测3D对象的周长 ST_perimeter3d(geometry)

量测两点构成的方位角, 单位弧度 ST_azimuth(geometry, geometry)

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何对象输出:

参考语义:

NDR: Little Endian

XDR: big-endian

HEXEWKB: Canonical

SVG: SVG 格式

GML: GML 格式

KML: KML 格式

GeoJson: GeoJson 格式

`ST_AsBinary(geometry, { 'NDR' | 'XDR' })`

`ST_AsEWKT(geometry)`

`ST_AsEWKB(geometry, { 'NDR' | 'XDR' })`

`ST_AsHEXEWKB(geometry, { 'NDR' | 'XDR' })`

`ST_AsSVG(geometry, [rel], [precision])`

`ST_AsGML([version], geometry, [precision])`

`ST_AsKML([version], geometry, [precision])`

`ST_AsGeoJson([version], geometry, [precision], [options])`

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何对象创建:

参考语义:

Dump: 转储 ST_GeomFromEWKT(text)

ST_GeomFromEWKB(bytea)
ST_MakePoint(, , [], [])
ST_MakePointM(, ,)
ST_MakeBox2D(,)
ST_MakeBox3D(,)
ST_MakeLine(geometry set)
ST_MakeLine(geometry, geometry)
ST_LineFromMultiPoint(multipoint)
ST_MakePolygon(linestring, [linestring[]])
ST_BuildArea(geometry)
ST_Polygonize(geometry set)
ST_Collect(geometry set)
ST_Collect(geometry, geometry)
ST_Dump(geometry)
ST_DumpRings(geometry)

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何对象编辑:

给几何对象添加一个边界, 会使查询速度加快 ST_AddBBOX(geometry)

删除几何对象的边界 ST_DropBBOX(geometry)

添加、删除、设置点 ST_AddPoint(linestring, point, [])

ST_RemovePoint(linestring, offset)

ST_SetPoint(linestring, N, point)

几何对象类型转换 ST_Force_collection(geometry)

ST_Force_2d(geometry)

ST_Force_3dz(geometry), ST_Force_3d(geometry),

ST_Force_3dm(geometry)

ST_Force_4d(geometry)

ST_Multi(geometry)

将几何对象转化到指定空间参考 ST_Transform(geometry,integer)

对3D几何对象作仿射变化 ST_Affine(geometry, float8, float8, float8, float8, float8, float8, float8, float8, float8, float8, float8, float8, float8)

对2D几何对象作仿射变化 ST_Affine(geometry, float8, float8, float8, float8, float8, float8)

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何对象编辑:

对几何对象作偏移 ST_Translate(geometry, float8, float8, float8)

对几何对象作缩放 ST_Scale(geometry, float8, float8, float8)

对3D几何对象作旋转 ST_RotateZ(geometry, float8)

ST_RotateX(geometry, float8)

ST_RotateY(geometry, float8)

对2D对象作偏移和缩放 ST_TransScale(geometry, float8, float8, float8, float8)

反转 ST_Reverse(geometry)

转化到右手定则 ST_ForceRHR(geometry)

参考IsSimple函数

使用Douglas-Peucker算法 ST_Simplify(geometry, tolerance)

ST_SimplifyPreserveTopology(geometry, tolerance)

讲几何对象顶点捕捉到网格 ST_SnapToGrid(geometry, originX, originY, sizeX, sizeY)

ST_SnapToGrid(geometry, sizeX, sizeY), ST_SnapToGrid(geometry, size)

第二个参数为点, 指定原点坐标 ST_SnapToGrid(geometry, geometry, sizeX, sizeY, sizeZ, sizeM)

分段 ST_Segmentize(geometry, maxlength)

合并为线 ST_LineMerge(geometry)

3.1 PostGIS空间函数

2. PostGIS扩展函数

线性参考：

根据location (0-1) 获得该位置的点 ST_line_interpolate_point(linestring, location)

获取一段线 ST_line_substring(linestring, start, end)

根据点获取location (0-1) ST_line_locate_point(LineString, Point)

根据量测值获得几何对象 ST_locate_along_measure(geometry, float8)

根据量测值区间获得几何对象集合 ST_locate_between_measures(geometry, float8, float8)

3.1 PostGIS空间函数

2. PostGIS扩展函数

杂项功能函数:

几何对象的摘要 ST_Summary(geometry)

几何对象的边界 ST_box2d(geometry) ST_box3d(geometry)

多个几何对象的边界 ST_extent(geometry set) 0=2d, 1=3dm, 2=3dz, 3=4d ST_zmflag(geometry)

是否包含Bounding Box ST_HasBBOX(geometry)

几何对象的维数: 2、3、4 ST_ndims(geometry)

子对象的个数 ST_nrings(geometry) ST_npoints(geometry)

对象是否验证成功 ST_isvalid(geometry)

扩大几何对象 ST_expand(geometry, float)

计算一个空间表的边界范围 ST_estimated_extent([schema], table, geocolumn)

获得空间参考 ST_find_srid(, ,)

几何对象使用的内存大小, 单位byte ST_mem_size(geometry)

点是否在圆上 ST_point_inside_circle(,,)

获取边界的X、Y、Z ST_XMin(box3d)

ST_YMin(box3d) ST_ZMin(box3d) ST_XMax(box3d) ST_YMax(box3d) ST_ZMax(box3d)

构造一个几何对象的数组 ST_Accum(geometry set)

3.1 PostGIS空间函数

2. PostGIS扩展函数

长事务支持:

启用/关闭长事务支持, 重复调用无副作用 `EnableLongTransactions()`

`DisableLongTransactions()`

检查对行的update和delete操作是否已授权 `CheckAuth([], , )`

锁定行 `LockRow([], , , [])`

解锁行 `UnlockRows()`

在当前事务中添加授权ID `AddAuth()`

3.2 PostGIS空间查询语言

nyc_census_blocks **人口普查块** (**census block**) 是人口普查数据的最小地理单位。

blkid	A 15-digit code that uniquely identifies every census block . Eg: 360050001009000
popn_total	Total number of people in the census block
popn_white	Number of people self-identifying as "White" in the block
popn_black	Number of people self-identifying as "Black" in the block
popn_nativ	Number of people self-identifying as "Native American" in the block
popn_asian	Number of people self-identifying as "Asian" in the block
popn_other	Number of people self-identifying with other categories in the block
boroname	Name of the New York borough. Manhattan, The Bronx, Brooklyn, Staten Island, Queens
geom	Polygon boundary of the block https://blog.csdn.net/qq_35732147

nyc_neighborhoods **社区**

name	Name of the neighborhood
boroname	Name of the New York borough. Manhattan, The Bronx, Brooklyn, Staten Island, Queens
geom	Polygon boundary of the neighborhood

nyc_streets **街道线**

name	Name of the street
oneway	Is the street one-way? "yes" = yes, "" = no
type	Road type (primary, secondary, residential, motorway)
geom	Linear centerline of the street

nyc_subway_stations **地铁站**

name	Name of the station
borough	Name of the New York borough. Manhattan, The Bronx, Brooklyn, Staten Island, Queens
routes	Subway lines that run through this station
transfers	Lines you can transfer to via this station
express	Stations where express trains stop, "express" = yes, "" = no
geom	Point location of the station

nyc_census_sociodata **人口普查区域对应的社会人口经济数据**

tractid	An 11-digit code that uniquely identifies every census tract . ("36005000100")
transit_total	Number of workers in the tract
transit_private	Number of workers in the tract who use private automobiles / motorcycles
transit_public	Number of workers in the tract who take public transit
transit_walk	Number of workers in the tract who walk
transit_other	Number of workers in the tract who use other forms like walking / biking
transit_none	Number of workers in the tract who work from home
transit_time_mins	Total number of minutes spent in transit by all workers in the tract (minutes)
family_count	Number of families in the tract
family_income_median	Median family income in the tract (dollars)
family_income_mean	Average family income in the tract (dollars)
family_income_aggregate	Total income of all families in the tract (dollars)
edu_total	Number of people with educational history
edu_no_highschool_dipl	Number of people with no high school diploma
edu_highschool_dipl	Number of people with high school diploma and no further education
edu_college_dipl	Number of people with college diploma and no further education
edu_graduate_dipl	Number of people with graduate school diploma https://blog.csdn.net/c

3.2 PostGIS空间查询语言

"查看布鲁克林所有社区的名字"

```
SELECT name  
FROM nyc_neighborhoods  
WHERE boroname = 'Brooklyn';
```

'West Village'社区 (neighborhood) 的面积是多少?

```
SELECT ST_Area(geom)  
FROM nyc_neighborhoods  
WHERE name = 'West Village';
```

"小意大利 (Little Italy) 社区"有什么地铁站? 它在哪些地铁线路上?

```
SELECT s.name, s.routes  
FROM nyc_subway_stations AS s  
JOIN nyc_neighborhoods AS n  
ON ST_Contains(n.geom, s.geom)  
WHERE n.name = 'Little Italy';
```


Shp vs postgis 查询语法对比



```
//构造一个面边界 node gdal
let _poly=gdal.Geometry.fromWKT(geom_wkt,srs);
//从数据集中查询与面边界相交的数据集
let datasets=[];
layer.features.forEach(function(feature, i) {
    //逐一判断图形是否相交
    if(feature.getGeometry().intersects(_poly))
        datasets.push(feature);
});
```

```
select * from t_test where st_intersects(geom,_poly); postgis
```

Sql多简单，为啥要写代码？



- 按名称分类统计我国各个省（面状图形）包含的城市数量：

```
SELECT a.name,count(b.id) FROM province_pg a, city_pt b
WHERE ST_Contains(a.geom, b.geom) group by a.name;
```

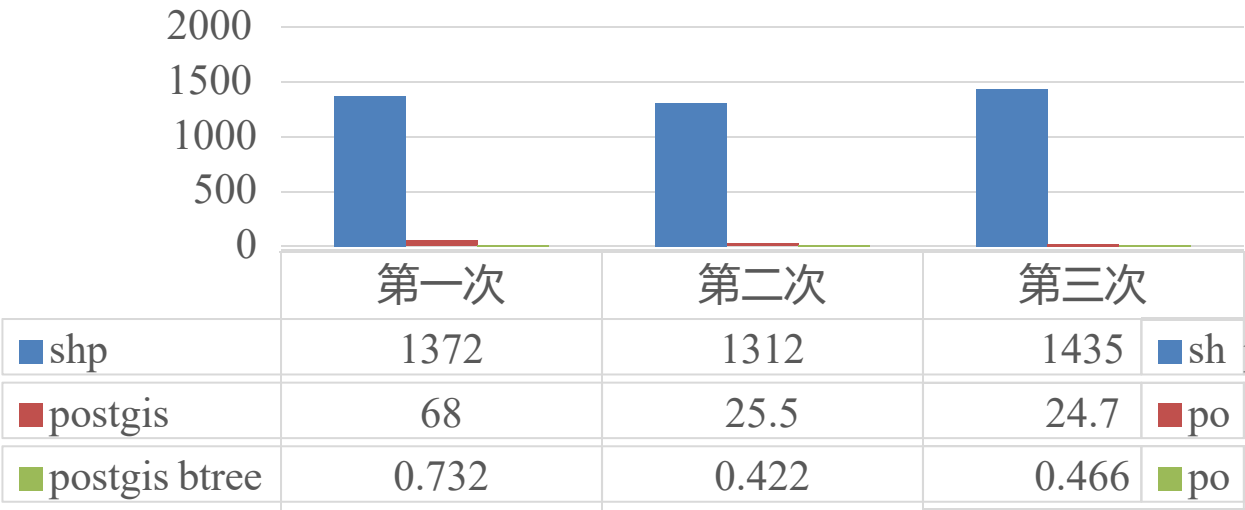
- 按名称分类统计缓冲区范围内受影响面状图形的面积：

```
SELECT name,count(name) num,sum(st_area(
    st_intersection(geom,buffergeom),true)
) as area from test_polygon where st_intersects(geom,buffergeom)
group by name;
```

Shp vs postgis 查询对比

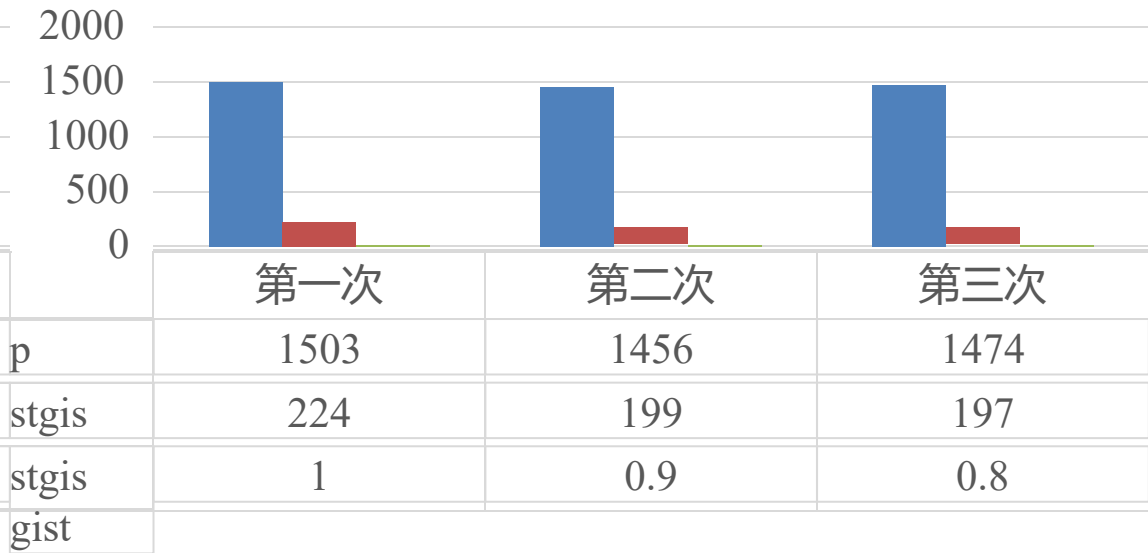
数据规模：191560

属性查询：name='练塘镇'
属性查询结果返回耗时（单位ms）

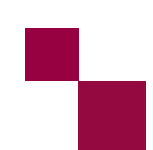


shp postgis postgis btree

空间查询：c:118.6,32.4,r:0.05缓冲
空间查询结果返回耗时（单位ms）



shp postgis postgis gist



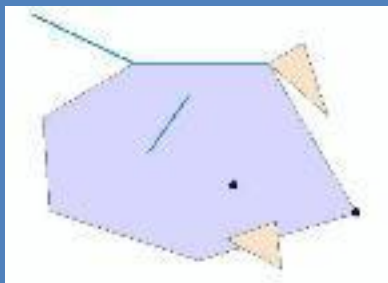
PostGIS数据处理中常见问题



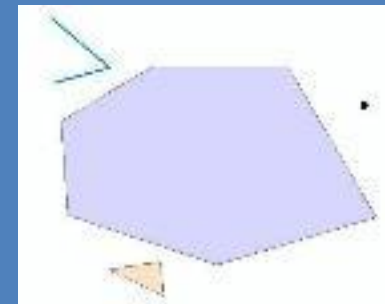
- 为什么我的查询语句慢到崩溃?
 - 1 SQL不走空间索引
 - 2 空间索引效率低
- 图形处理方案没有好的思路



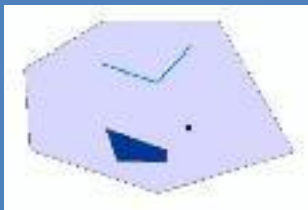
空间关系简介



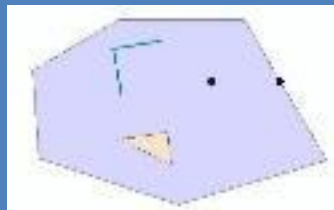
ST_Intersects相交



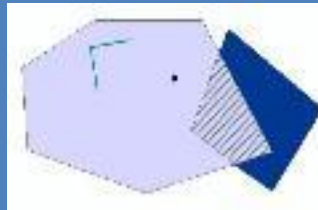
ST_Disjoint相
离



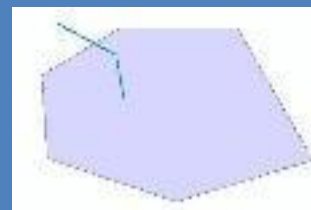
ST_Contains
ST_Within
包含/被包含



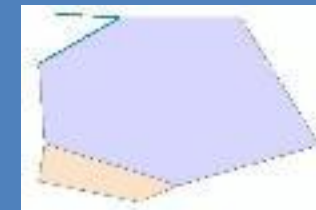
ST_Cover
S
覆盖



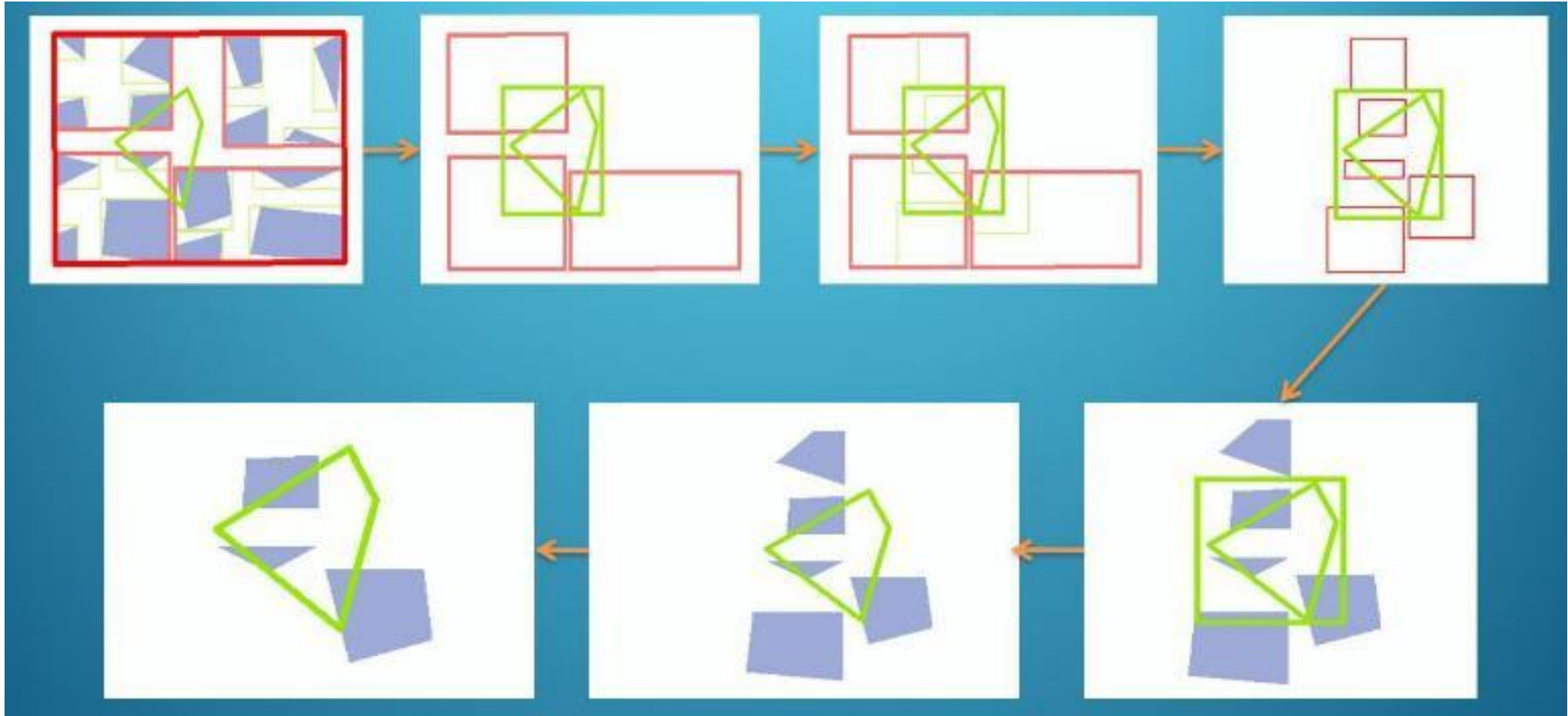
ST_Overlap
S
压盖



ST_Crosse
S
穿越



ST_Touche
S
相连



问题一 SQL不走空间索引

```
create table poi(  
  gid serial primary key,    --自增主键  
  name text,                 --属性字段  
  geom geometry(Point,4326) --图形字段  
);  
--建立空间索引  
create index poi_geom_idx on poi using gist(geom);
```

```
--生成250万测试点数据  
insert into poi(geom) SELECT st_setsrid((ST_Dump(p_geom)).geom,4326) from (  
  select ST_GeneratePoints(  
    ST_GeomFromText('Polygon((113 24,113 35,125 35,125 24,113 24))'),  
    2500000) as p_geom  
  ) as b;
```

问题：如何查询当前“我的位置 (118.1249,32.0987)”附近一公里内的所有poi?



Where后geom字段使用表达式函数

- 查询示例语句1:

```
Explain select a.* from poi a where  
    ST_Intersects(ST_Buffer(a.geom,0.01),  
    ST_SetSrid(ST_MakePoint(118.3749,32.0987),4326));  
Parallel Seq Scan on poi a ...  
Filter: st_intersects(st_buffer(geom, 0.01), pt)
```



```
Explain select a.* from poi a where ST_Intersects(a.geom,  
st_buffer(ST_SetSrid(ST_MakePoint(118.3749,32.0987),4326),0.0  
1));  
Index Scan using poi_geom_idx on poi a (actual time=0.065..0.104 rows=5 loops=1)
```

Where后geom字段使用表达式函数

- 查询示例语句2:

```
select a.* from poi a where ST_Distance(a.geom,  
    ST_SetSrid(ST_MakePoint(118.3749,32.0987),4326))<0.0  
    1;
```

Parallel Seq Scan on poi a (actual time=307.310..**563.545** rows=5 loops=1) --顺序扫描
Filter: (st_distance(geom, pt) < 0.01)



```
explain(analyse) select a.* from poi a where ST_DWithin(a.geom,  
    ST_SetSrid(ST_MakePoint(118.3749,32.0987),4326),0.0  
    1);
```

Index Scan using idx on poi a(actual time=0.066..**0.094** rows=5 loops=1) --索引扫描

- 总结: where后对geom使用表达式会导致索引失效。

特殊Function改变geom类型

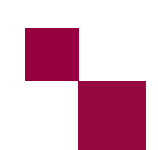
- 查询示例语句:

```
explain(analyse) select a.* from poi a where ST_DWithin(a.geom::geography,  
    ST_SetSrid(ST_MakePoint(118.3749,32.0987),4326)::geography,100  
    0);
```

Seq Scan on poi a (actual time=700.502..744.216 rows=7 loops=1)

```
Create index poi_geogra_idx on poi using gist((geom::geography));  
explain(analyse) select a.* from poi a where ST_DWithin(a.geom::geography,  
    ST_SetSrid(ST_MakePoint(118.3749,32.0987),4326)::geography,100  
    0); Gather (actual time=1.116..33.130 rows=7 loops=1)  
-> Parallel Bitmap Heap Scan ...->Bitmap Index Scan on poi_gra_idx  
(actual time=0.439..0.439 rows=9 loops=1)
```

- 总结：特殊函数导致geom类型改变不走索引，适时可引入表达式索引。



小结



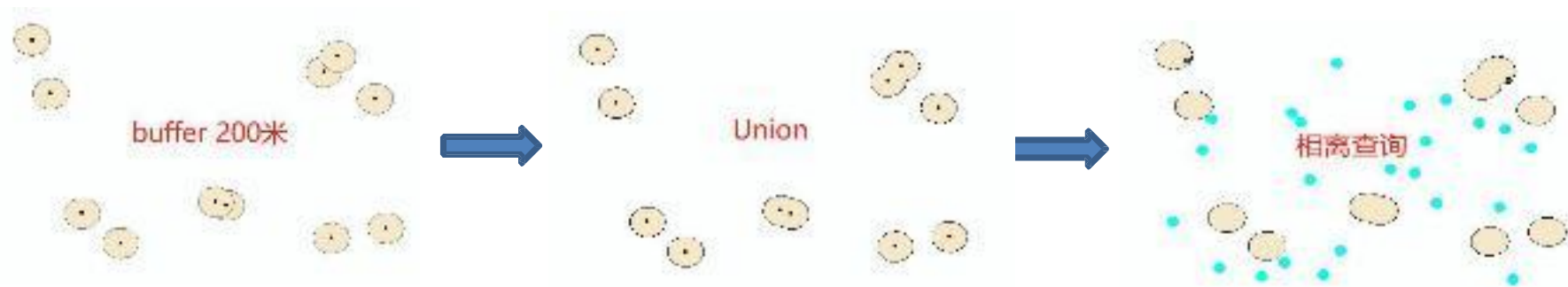
- 1 PostGIS解决同一个问题的写法有多种，需要仔细审核与对比。
- 2 不在where后对geom字段进行函数计算或类型转换操作。
- 3 特殊情况可建立表达式索引。



查询使用ST_Disjoint

- 场景介绍：已知居民点house表数量25000，交通点station表数量6000，统计所有交通点200米以外的居民点数量。

查询逻辑：1 station先缓冲200米；2 合并所有小缓冲区为一个面；3 计算与缓冲面不相交的居民点；

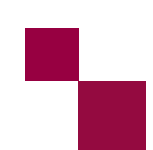


• 查询示例语句：

```
test=# select count(a.*) from house a,(  
select st_union(st_buffer(geom::geography,200)::geometry) geom from station  
) b where ST_Disjoint(a.geom,b.geom);  
^CCancel request sent  
ERROR: canceling statement due to user request  
Time: 3504594.932 ms (58:24.595)
```



```
explain(analyse) select count(a.*) from house a where gid not in  
    ( select distinct(a.gid) from house a,  
      (select st_union(st_buffer(geom::geography,200)::geometry) geom from station) b  
      where ST_intersects(a.geom,b.geom) );  
Aggregate (actual time=36163.208..36163.208 rows=1 loops=1) (总耗时36s)  
  -> Seq Scan on house ...  
    -> Index Scan using house_geom_idx on house a_1  
      (actual time=38.881..31971.891 rows=1453 loops=1) ... (空间索引耗时32s)  
        Rows Removed by Filter: 23547 (查询点总数25000, 索引扫描到无效数据23547)
```



ST_Disjoint优化小结



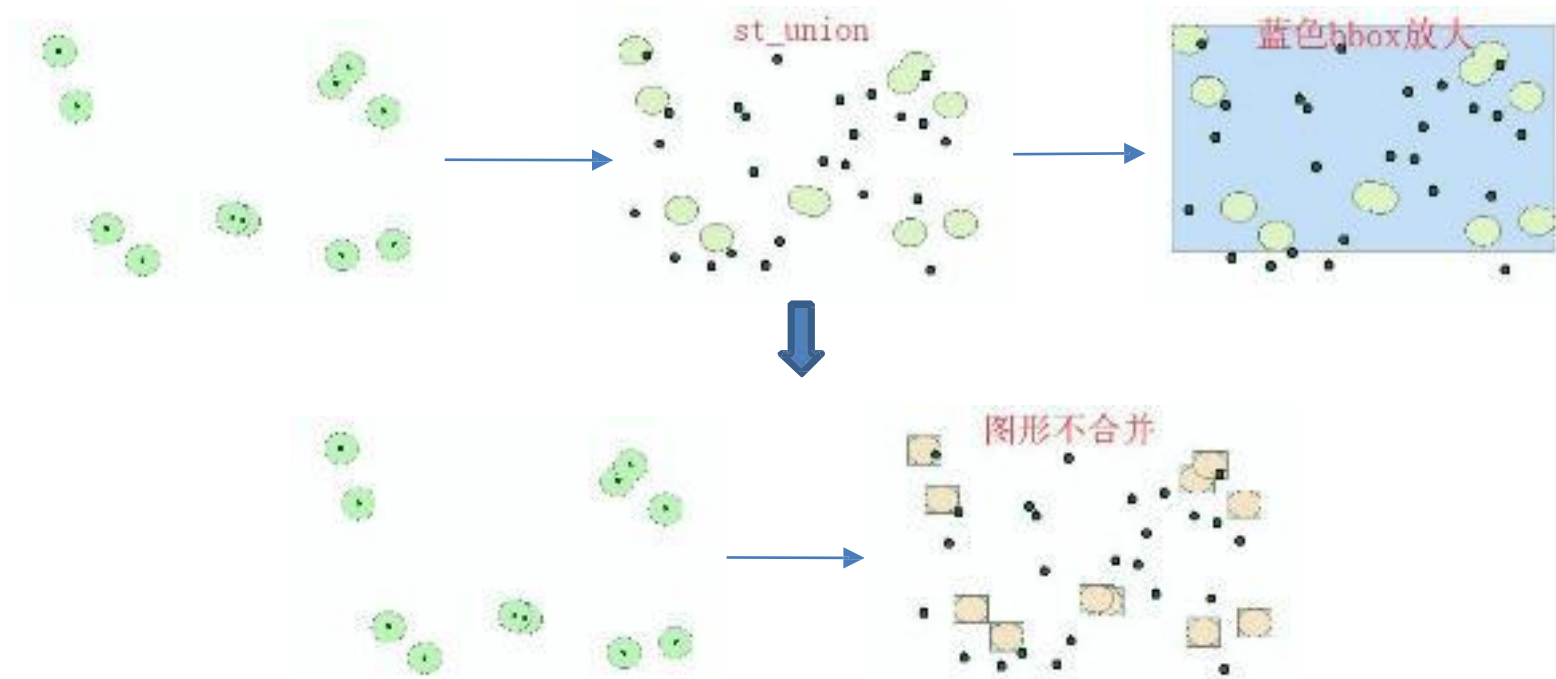
- 1 ST_Disjoint（相离）函数不符合rtree索引的查询机制，绝对不会走索引。
- 2 ST_Disjoint函数仅适用于where前判定空间关系的结果，不可用于where后根据空间关系查询的条件。
- 3 ST_Disjoint优化应采用“ST_Intersects（相交）取反”方案。

遗留问题：空间索引耗时32s/总耗时36s，索引效果太差。



问题二 空间索引效率低

- 案例1.3索引效率低图示：



图形小，bbox大，会导致索引扫描范围放大，效率变差



where后或者子查询中使用ST_Union

查询示例语句：

- select count(a.*) from house a where gid not in
(select distinct(a.gid) from house a,
- (select ST_Union(st_buffer(geom::geography,200)::geometry) geom from station) b where
ST_intersects(a.geom,b.geom)); --36s

Union等图形处理函数
应该只在输出结果用，
不能在查询条件里用！

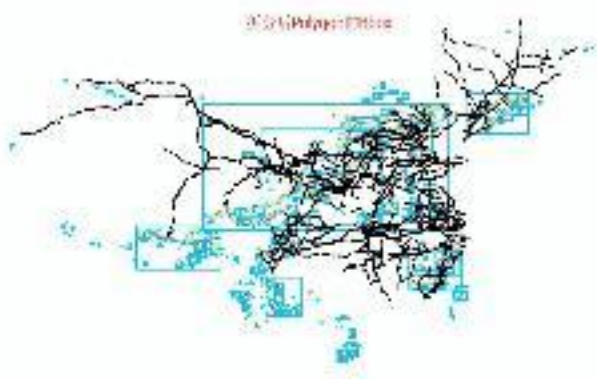
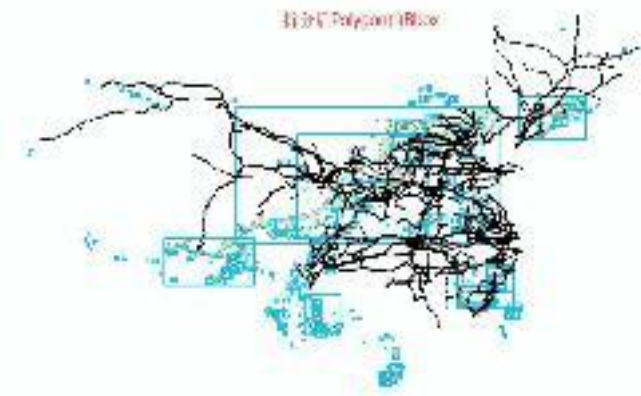
select count(a.*) from house a where gid not in
(select distinct(a.gid) from house a,
(select st_buffer(geom::geography,200)::geometry geom from station) b
where ST_intersects(a.geom,b.geom)); --0.73s



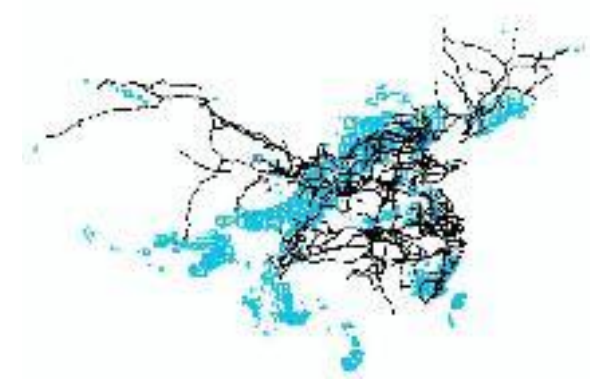
geom对象本身太复杂



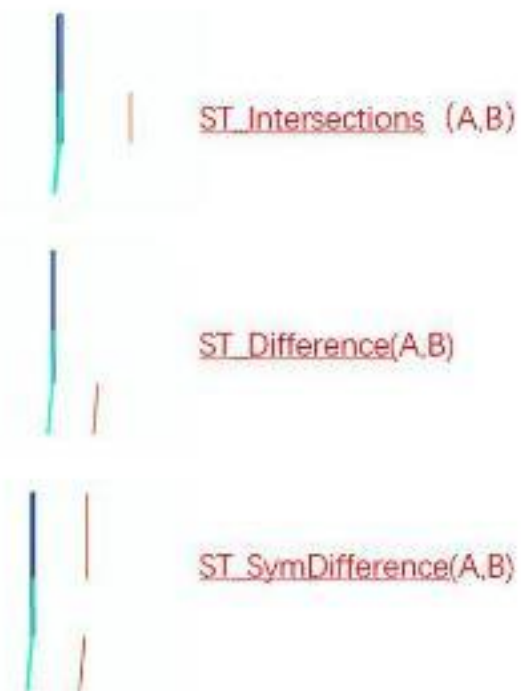
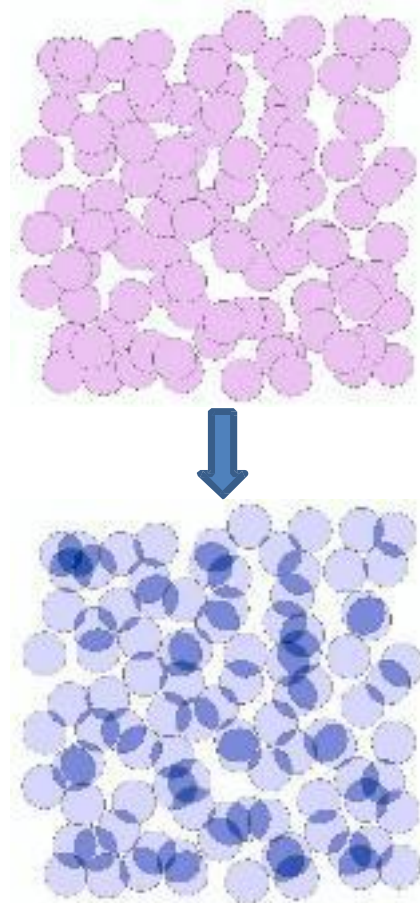
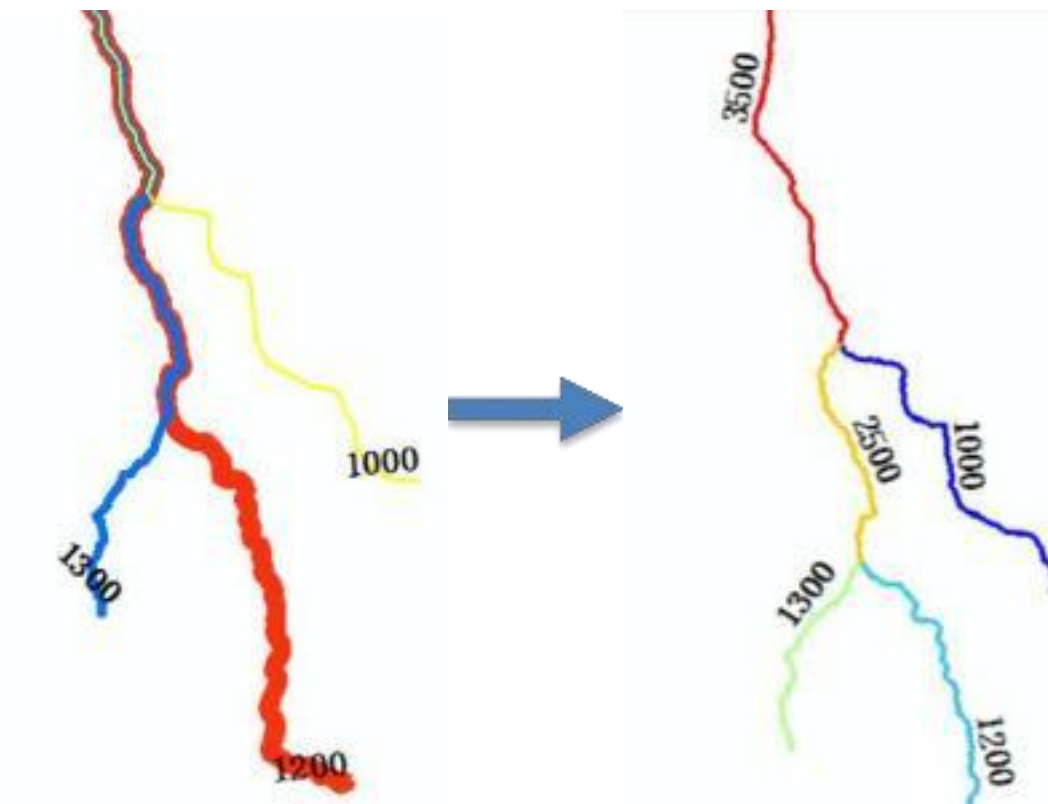
MultiPolygon Polygon
ST_NumGeometries(geom)
ST_GeometryN(geom,N)



Polygon 小Polygon集合
ST_SubDivide(geom,n)



问题三 复杂的图形效果没有思路



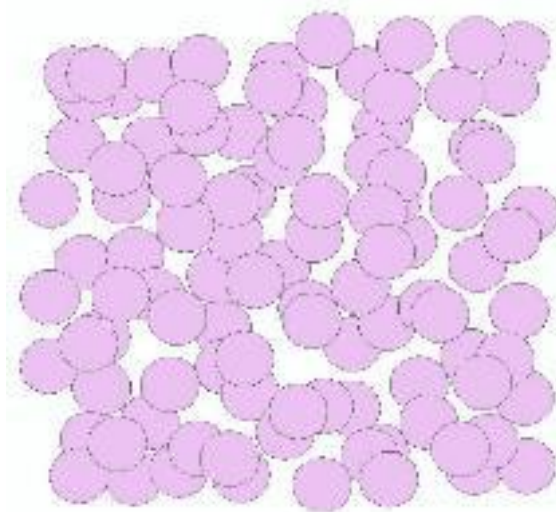
面压盖处理 面拆线



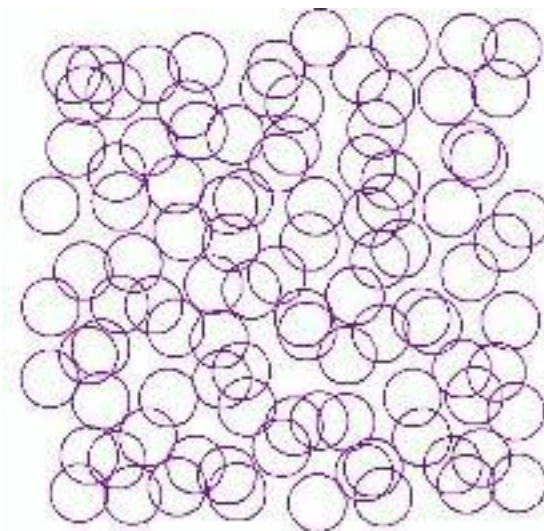
Create table boundaries as

```
Select st_union(st_exteriorRing(geom)) as geom from circles;
```

-- st_exteriorRing提取面的边界



面拆线

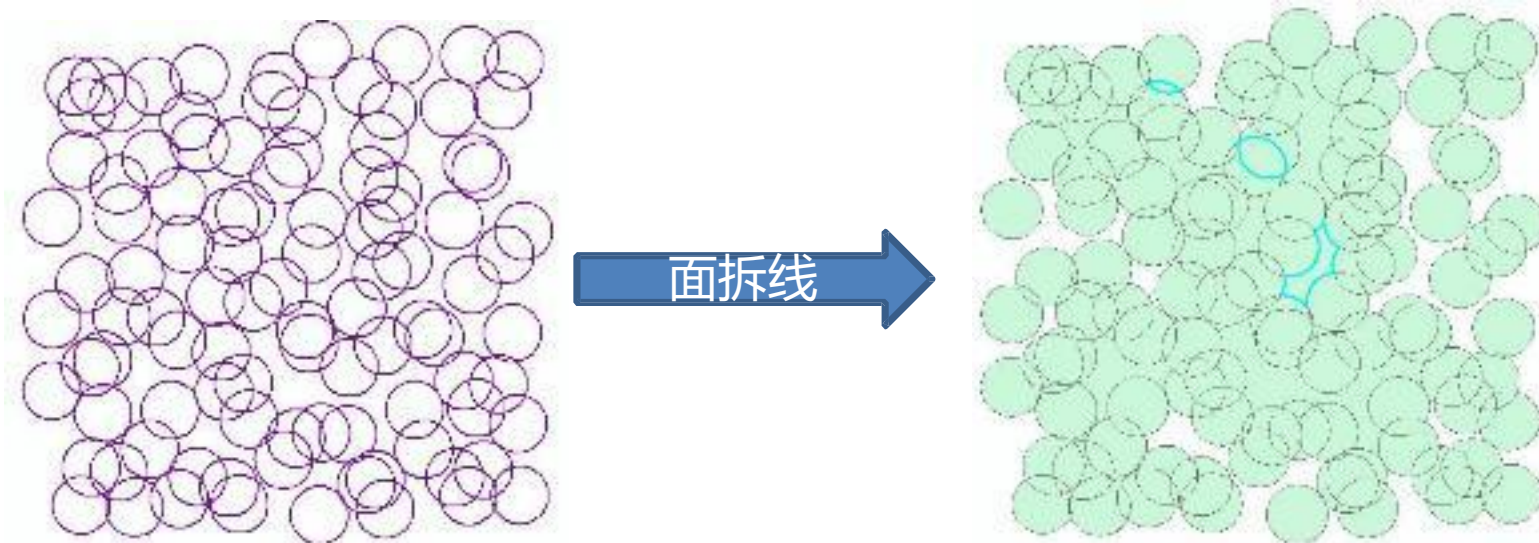


面压盖处理 线构面



```
createtablepolysasselectnextval('polyseq')asid, (st_dump(st_polygonize(geom))).geom as geom from boundaries;
```

-- st_polygonize根据线坐标自动构造面，首尾不闭合的线无法构面会舍弃

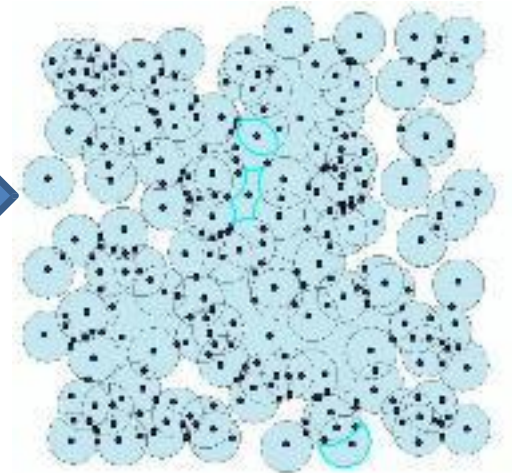


面压盖处理 更新面的权重

```
alter table polys add column count integer default 0;

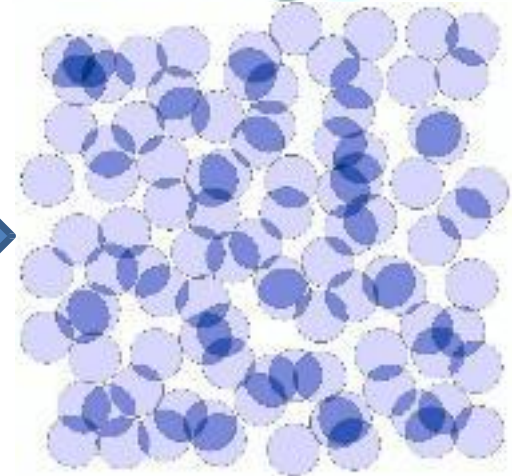
update polys set count = p.count from (select count
(*) as count, p.id as id      from polys p join circles
on ST_Contains(c.geom, ST_PointOnSurface(p.g
eom))) group by      p.id ) AS p where p.id = polys.id;
```

绑定权重



```
delete from POLYS where count=0;
```

剔除冗余
分级渲染



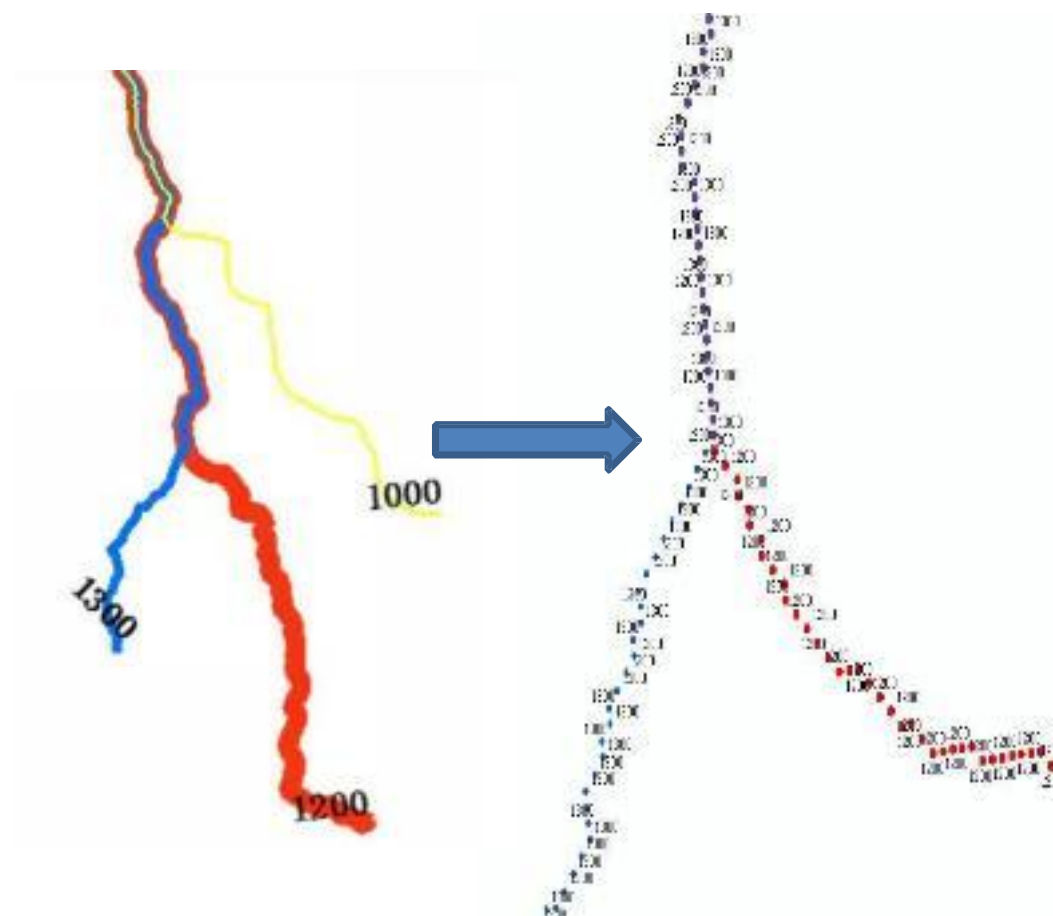
线压盖处理 线拆点

```
SELECT edge_id, (dp).path[1] As index, ST_AsText((dp).geom) As wktnode
FROM (SELECT 1 As edge_id
      , ST_DumpPoints(ST_GeomFromText('LINESTRING(1 2, 3 4, 10 10)')) AS dp
      UNION ALL
      SELECT 2 As edge_id
      , ST_DumpPoints(ST_GeomFromText('LINESTRING(3 5, 5 6, 9 10)')) AS dp
      ) As foo;
```

edge_id	index	wktnode
1	1	POINT(1 2)
1	2	POINT(3 4)
1	3	POINT(10 10)
2	1	POINT(3 5)
2	2	POINT(5 6)
2	3	POINT(9 10)



```
CREATE TABLE pt(
  gid serial primary key,
  line_id integer, --点位的所属线路id
  weight numeric, --点位权重, 与其所属线路一致
  ptindex int, --点位在其所属线路中的顺序
  geom geometry(Point,4326)
);
```



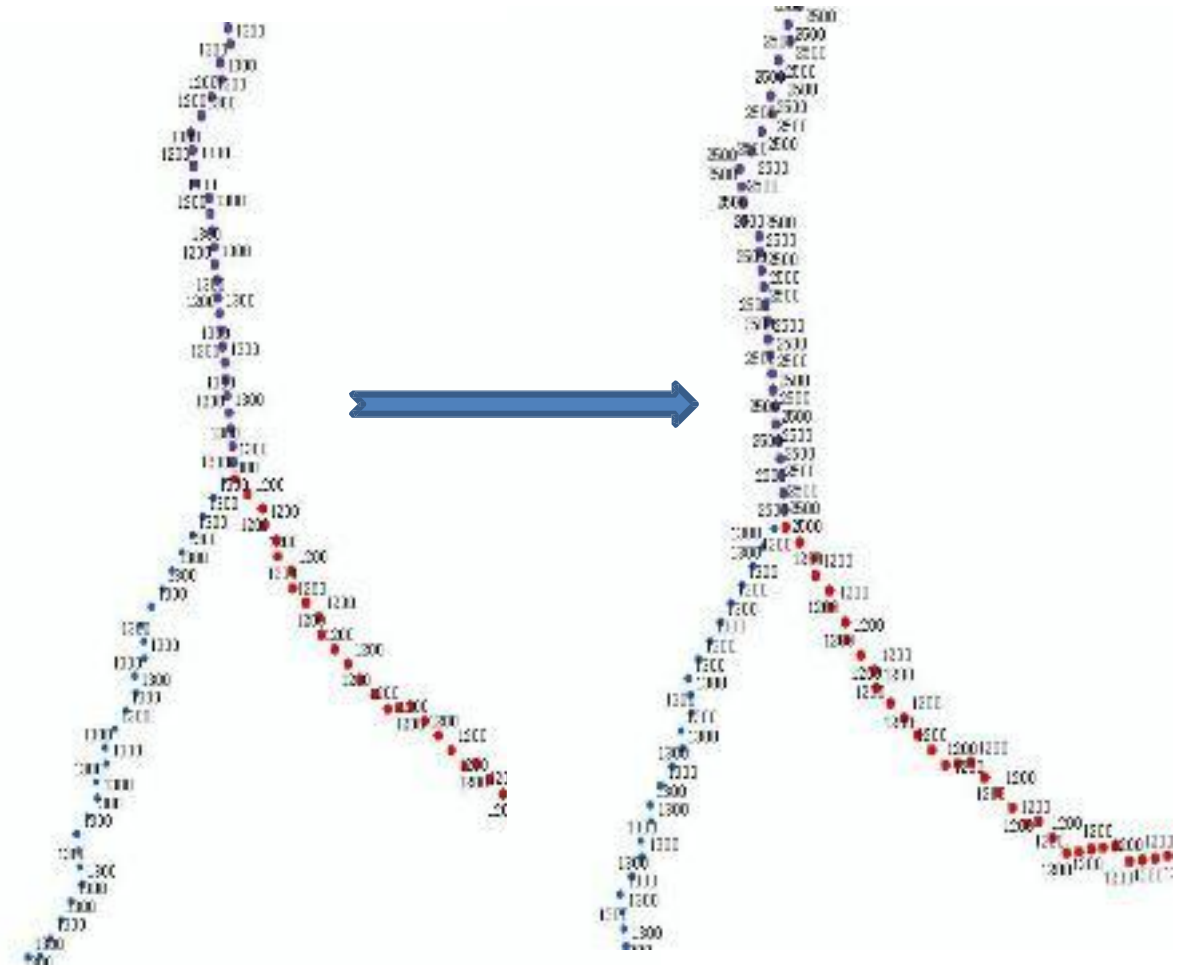
线压盖处理 点位聚合权重更新

```
select array_agg(gid) gids, sum(weight) weight
into pt_group from pt group by geom;
```



```
create index pt_group_gids_idx on pt_group
using gin(gids);
```

```
update pt t2 set weight=t1.weight from
pt_group t1 where t2.gid=any(t1.gids);
```



线压盖处理 点连线剔除冗余



--根据点的线路id, 线路中的顺序号排序

```
For rec in select * from pt order by line_id,ptindex loop
```

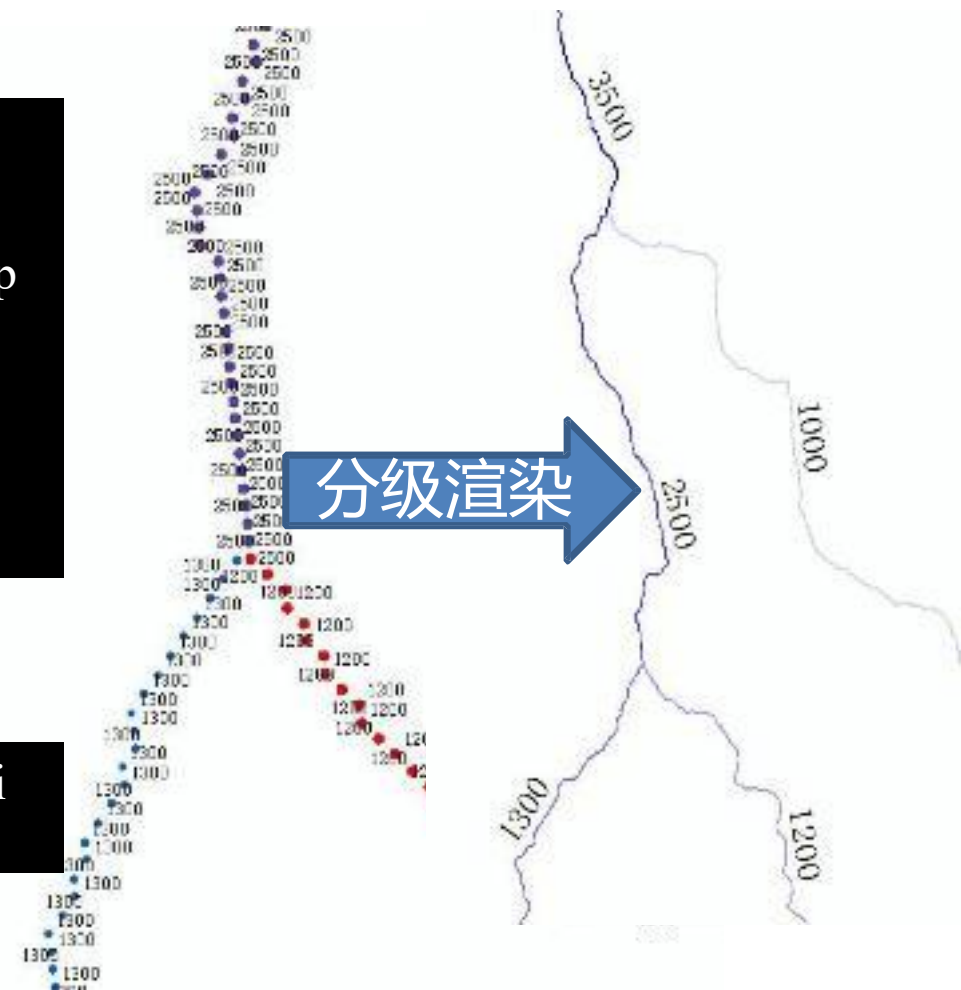
--权重连续的点构成一个line

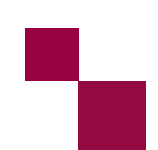
```
End loop;
```

↓
去重

```
Delete from line_result where gid not in (select min(cti  
d) from line_result group by geom)
```

分级渲染





总结



- 1 PostGIS入门简单，功能全面。
- 2 开发敏捷，试错与调优效率高。
- 3 熟练掌握PostGIS的函数是解决复杂综合问题的前提。
- 4 深入理解PG的索引机制及其对应场景是调优的关键。
- 5 非常适合科研机构做GIS数据处理平台，服务发布数据源。



谢谢大家!

